AI Assisted Coding
Assignment – 19.2

Name: Mohammed Rahil Raza

Ht.no: 2303A51433

bt.no: 21

Task 1 – Python to Java Conversion

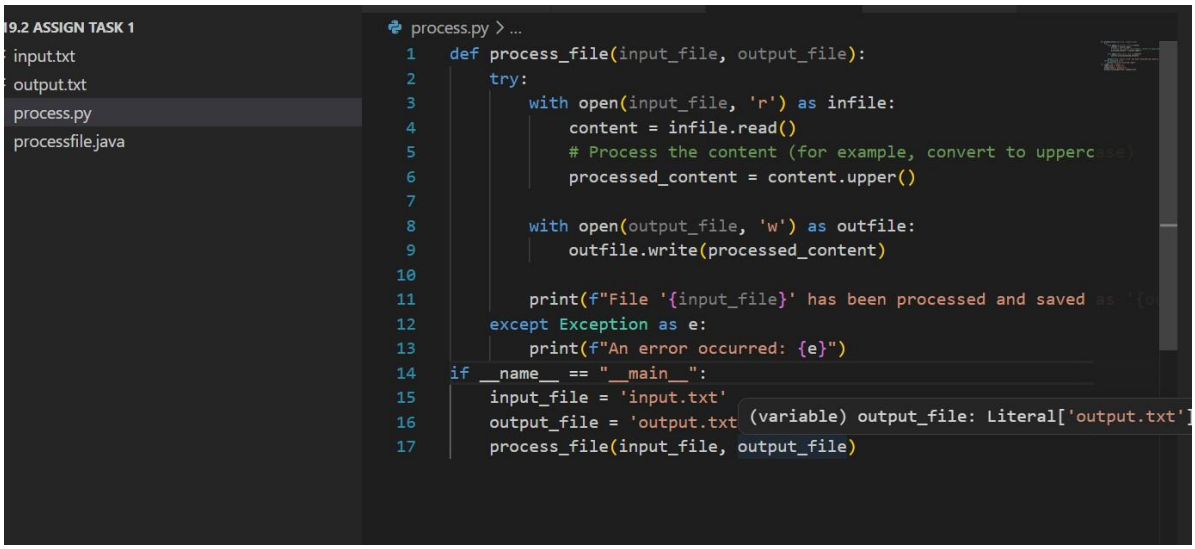
Task:
Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

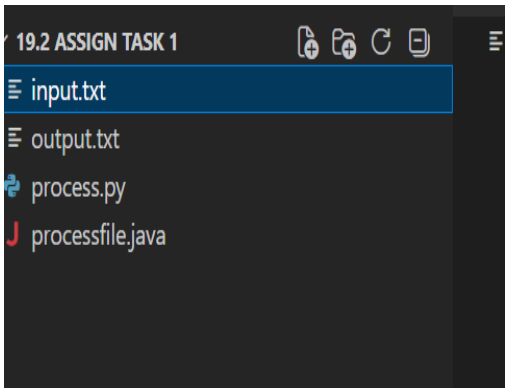
Instructions:

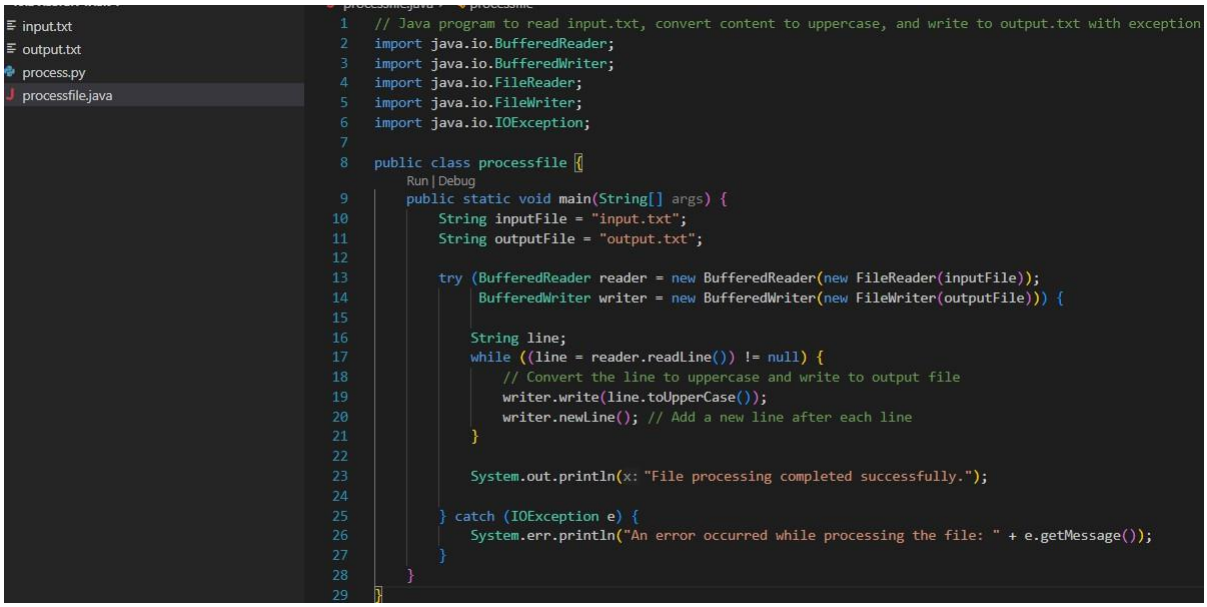
- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

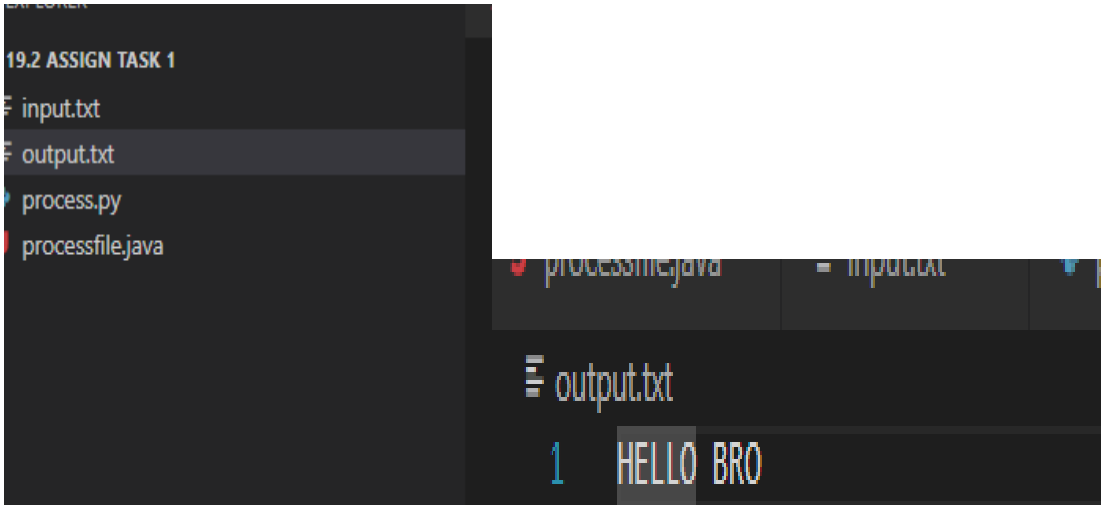
Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.









Task 2 – C++ to Python Function Conversion

Task:
Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Code :

```
bubble.cpp > ...
#include <iostream>
#include <vector>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
void printArray(const vector<int>& arr) {
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
}
int main() {
    // Test case 1: Basic test case
    vector<int> arr1 = {64, 34, 25, 12, 22, 11, 90};
    cout << "Original array: ";
    printArray(arr1);
    bubbleSort(arr1);
    cout << "Sorted array: ";
    printArray(arr1);

    // Test case 2: Already sorted array
    vector<int> arr2 = {1, 2, 3, 4, 5};
    cout << "\nOriginal array: ";
    printArray(arr2);
    bubbleSort(arr2);
    cout << "Sorted array: ";
    printArray(arr2);

    // Test case 3: Reverse sorted array
    vector<int> arr3 = {5, 4, 3, 2, 1};
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE
```

```
File  Edit  Selection  View  ...  ←  →  19.2
RUN AND DEBUG
RUN
Run and Debug
To customize Run and Debug create a launch.json file.
Debug using a terminal command or in an interactive chat.
bubble.cpp 2   bubble.py
C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\19.2 task 2\bubble.cpp • 2 problems in this file
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
# Example usage
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    bubble_sort(arr)
    print("Sorted array:", arr)
```

Task 3 – SQL to Pandas Query

Task:
Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

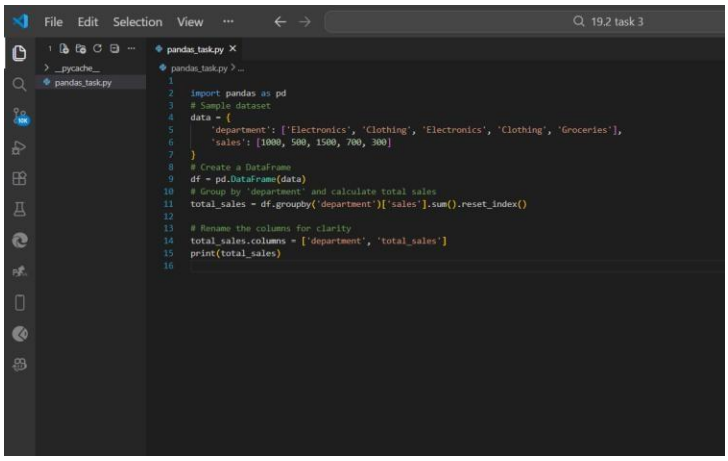
- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).

- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

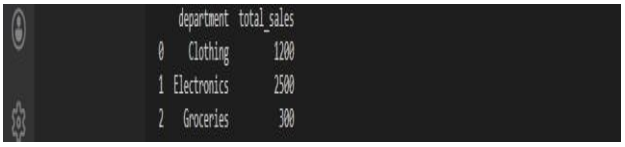
Code:



```python
import pandas as pd
# Sample dataset
data = {
    'department': ['Electronics', 'Clothing', 'Electronics', 'Clothing', 'Groceries'],
    'sales': [1000, 500, 1500, 700, 300]
}
# Create a DataFrame
df = pd.DataFrame(data)
# Group by 'department' and calculate total sales
total_sales = df.groupby('department')['sales'].sum().reset_index()

# Rename the columns for clarity
total_sales.columns = ['department', 'total_sales']
print(total_sales)
```

Output:



```
   department  total_sales
0    Clothing         1200
1 Electronics         2500
2   Groceries          300
```

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Code:

python.py > ...

```python
# Calculate total and average marks

students = [
    {"name": "A", "marks": [80, 90, 85]},
    {"name": "B", "marks": [70, 60, 75]},
    {"name": "C", "marks": [88, 92, 95]}
]

for student in students:
    total = sum(student["marks"])
    avg = total / len(student["marks"])

    print(f"{student['name']} -> Total: {total}, Average: {avg:.2f}")
```

```cpp

#include <iostream>
using namespace std;
void displayMenu() {
    cout << "Simple Calculator" << endl;
    cout << "1. Addition" << endl;
    cout << "2. Subtraction" << endl;
    cout << "3. Multiplication" << endl;
    cout << "4. Division" << endl;
    cout << "5. Exit" << endl;
}
int main() {
    int choice;
    double num1, num2;

    do {
        displayMenu();
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 4) {
            cout << "Enter two numbers: ";
            cin >> num1 >> num2;
        }

        switch (choice) {
            case 1:
                cout << "Result: " << num1 + num2 << endl;
                break;
            case 2:
                cout << "Result: " << num1 - num2 << endl;
                break;
            case 3:
                cout << "Result: " << num1 * num2 << endl;
                break;
            case 4:
                if (num2 != 0) {
                    cout << "Result: " << num1 / num2 << endl;
                } else {
                    cout << "Error: Division by zero is not allowed." << endl;
                }
                break;
            case 5:
                cout << "Exiting the calculator. Goodbye!" << endl;
                break;
            default:
                cout << "Invalid choice. Please try again." << endl;
```

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

python.py > ...

```python

def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
number = 10
result = check_even_odd(number)
print(f"The number {number} is {result}.")
number = 15
result = check_even_odd(number)
print(f"The number {number} is {result}.")

```

cpp.cpp 2

cpp.cpp > ...

```cpp
#include <iostream>
using namespace std;
bool isEven(int number) {
    return number % 2 == 0;
}
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (isEven(num)) {
        cout << num << " is even." << endl;
    } else {
        cout << num << " is odd." << endl;
    }

    return 0;
}

```

Output:

```
The number 10 is Even.
The number 15 is Odd.
```